COMSATS University Islamabad, Lahore Campus
Department of Computer Sciences

Terminal Lab – Semester Spring 2026

Course Title	Software Testing				Course Code	CSE482	Credit Hours: 3
Course Instructor	Ms. Yella Mehroze				Program Name	BSE	
Semester	8th	Batch:	FA22-BSE	Section	A	Type	Theory
Total Marks	50	Obtained Marks:			Date	May 09, 2026	
Student's Name	**UMAIR ALI**				Reg. No	FA22-BSE-137	

Question1: ***[Marks 20]***

[CLO <5>]: Create test plan document for a medium size software system in a team environment.

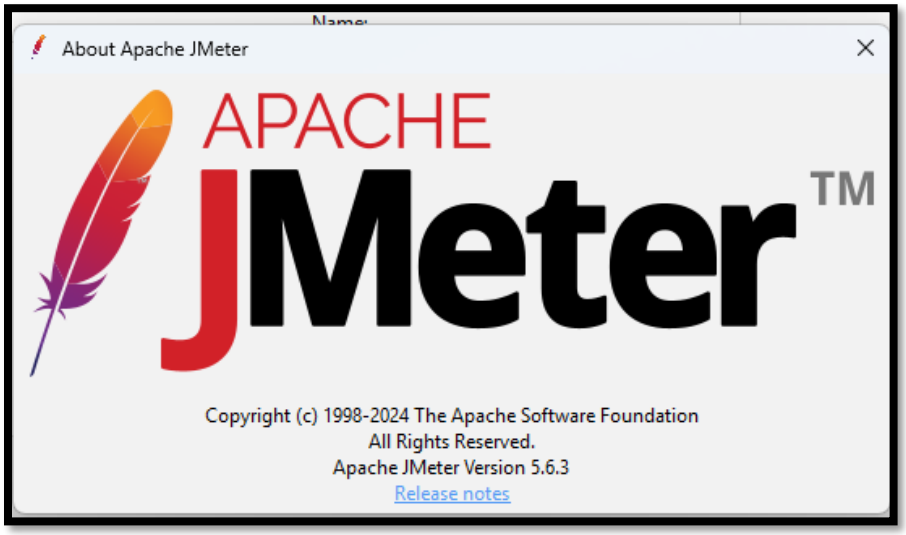
Bloom Taxonomy Level<Create>

Part A - Choose the Apache JMeter to perform performance Testing.

INSTALLATION — DO THIS FIRST

1. Install Java JDK 11+ → https://adoptium.net
2. Download JMeter 5.6+ → https://jmeter.apache.org/download_jmeter.cgi
3. Extract to a folder → e.g. C:\JMeter or /opt/jmeter
4. Launch JMeter GUI → Windows: bin\jmeter.bat | Linux/Mac: ./bin/jmeter.sh
5. Confirm installation → Help menu → About Apache JMeter → must show version 5.6+

```
┌─[ SWEET ]─[ SUNDAY 17/05/2026 - 5:02:53 PM ]─[ C:\Portable\AST-Terminal ]
└─[ UMAIR •• PowerShell ] :]  java --version
openjdk 25.0.3 2026-04-21 LTS
OpenJDK Runtime Environment Microsoft-13877124 (build 25.0.3+9-LTS)
OpenJDK 64-Bit Server VM Microsoft-13877124 (build 25.0.3+9-LTS, mixed mode, sharing)
```



TASK 1 Foundations — GUI, HTTP Requests & Assertions

Covers: Installation · Thread Groups · HTTP Methods · Assertions

This task builds the complete foundation of JMeter. By the end you will understand the GUI, be able to send all types of HTTP requests, and validate responses using assertions.

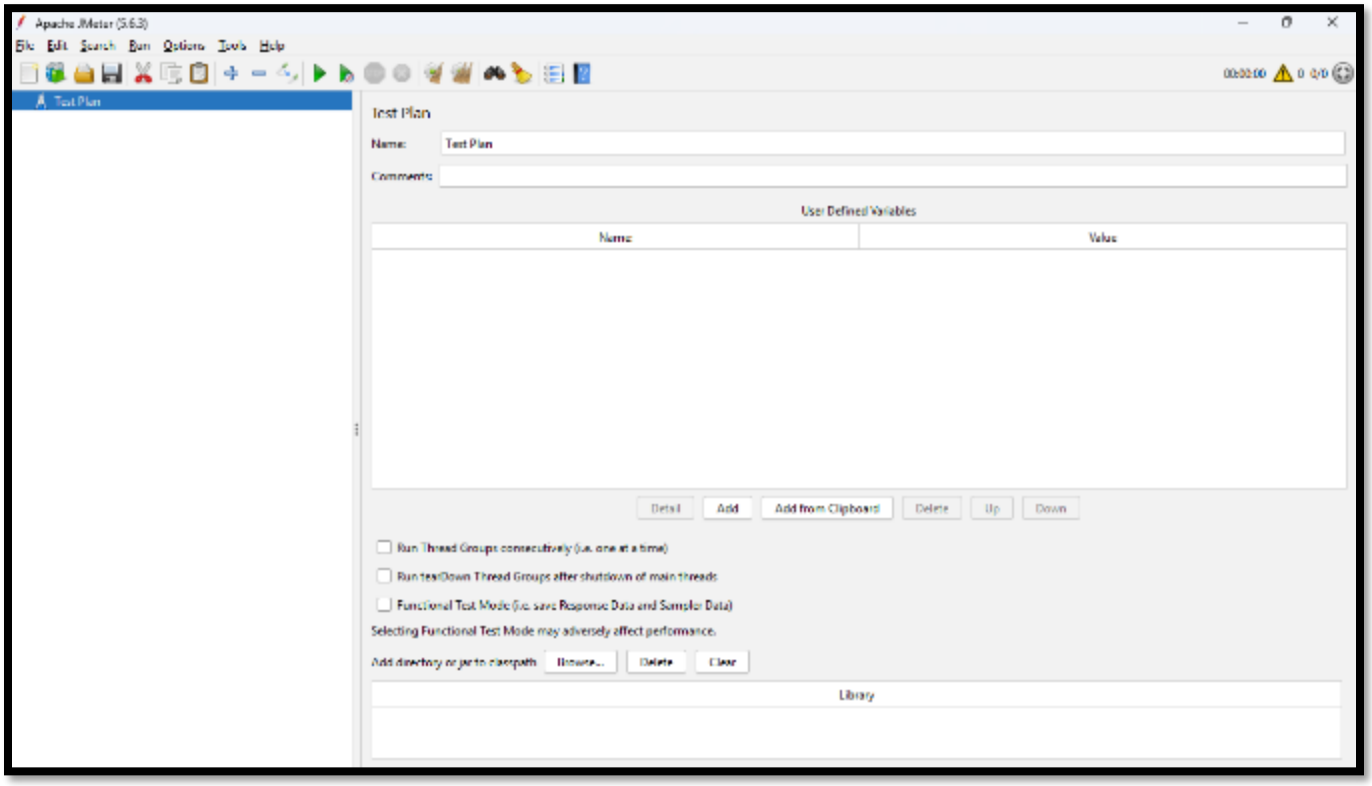
Part A — GUI Orientation & First Request

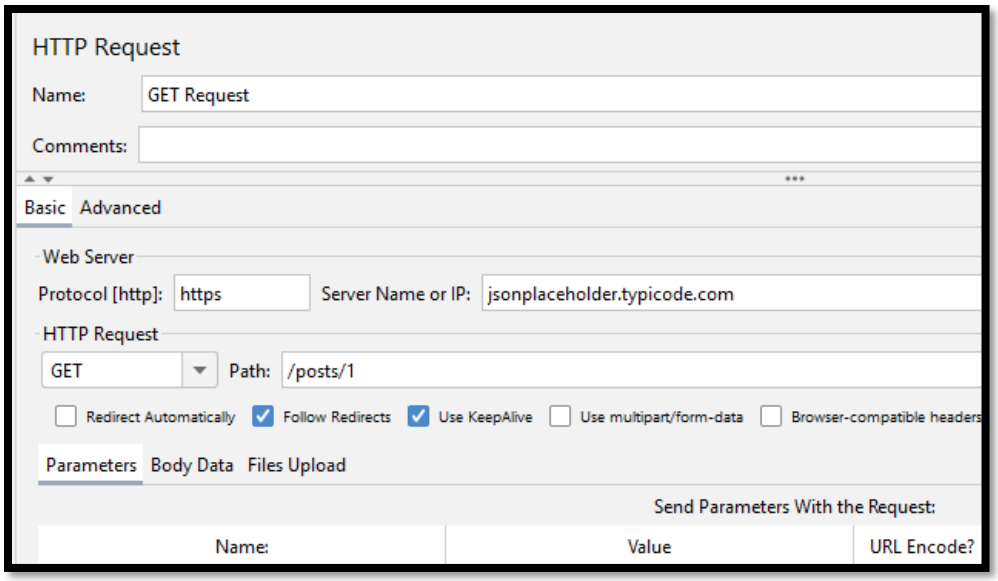
After installing JMeter, explore the interface and send your first HTTP request to a live API.

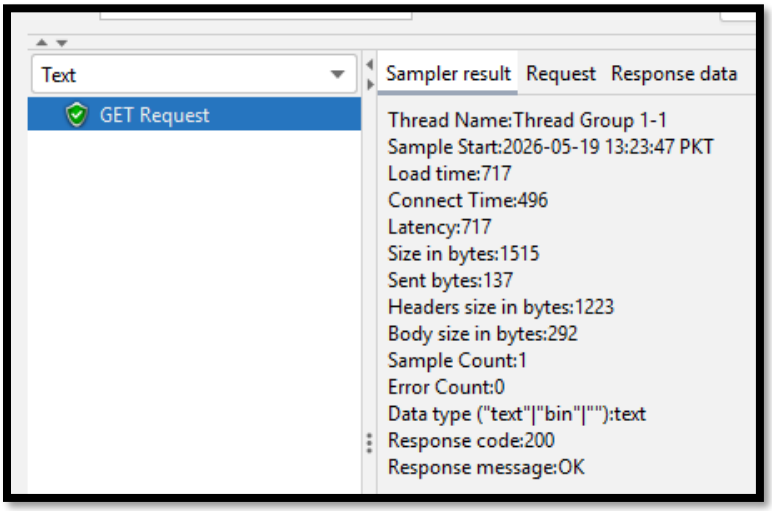
1. Launch JMeter and identify these 5 components: Test Plan, Menu Bar, Toolbar, Tree Panel (left), Detail Panel (right)
2. Right-click Test Plan → Add → Threads → Thread Group. Set: 1 user, Ramp-Up 1s, Loop Count 1
3. Right-click Thread Group → Add → Sampler → HTTP Request. Configure as follows:

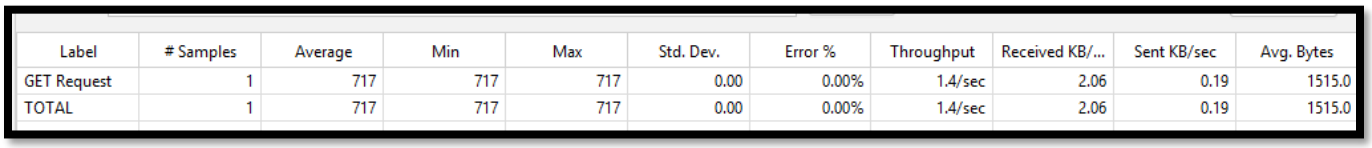
```
Protocol:    https
Server Name: jsonplaceholder.typicode.com
Method:      GET
Path:        /posts/1
```

4. Add two listeners: View Results Tree and Summary Report
5. Click the green Run button. Confirm a GREEN result. Note the response time and response code









Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/...	Sent KB/sec	Avg. Bytes
GET Request	1	717	717	717	0.00	0.00%	1.4/sec	2.06	0.19	1515.0
TOTAL	1	717	717	717	0.00	0.00%	1.4/sec	2.06	0.19	1515.0

```
Response Time:  2.06/sec received; 0.19/sec sent
Response Code:  200
```

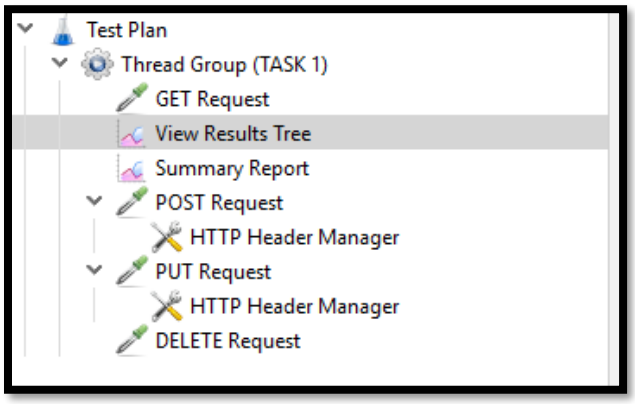
B — All HTTP Methods

Extend your test plan to cover all four HTTP methods used in modern REST web APIs.

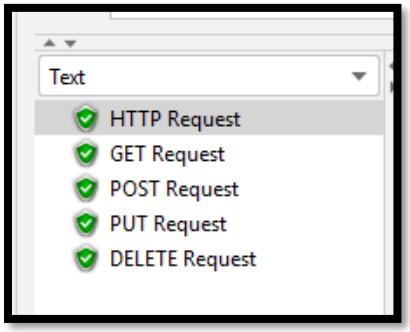
6. Add 3 more HTTP Request samplers to the same Thread Group for POST, PUT, and DELETE:

```
POST   /posts
  Body: {"title":"Test Post","body":"Hello World","userId":1}

PUT    /posts/1
  Body: {"id":1,"title":"Updated Title","body":"Updated Body","userId":1}

DELETE /posts/1
  (no body needed)

For POST and PUT — add HTTP Header Manager to each sampler:
  Right-click sampler → Add → Config Element → HTTP Header Manager
  Name: Content-Type    Value: application/json
```

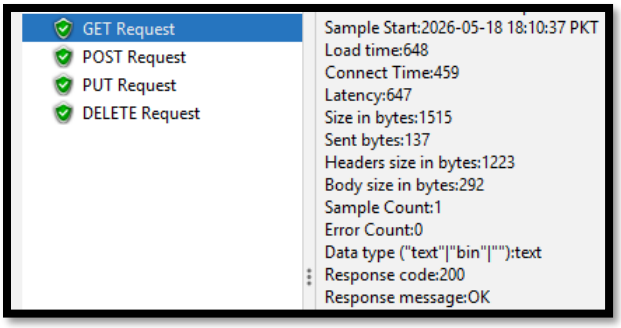
7. Run all 4 samplers. Expected response codes: GET = 200, POST = 201, PUT = 200, DELETE = 200



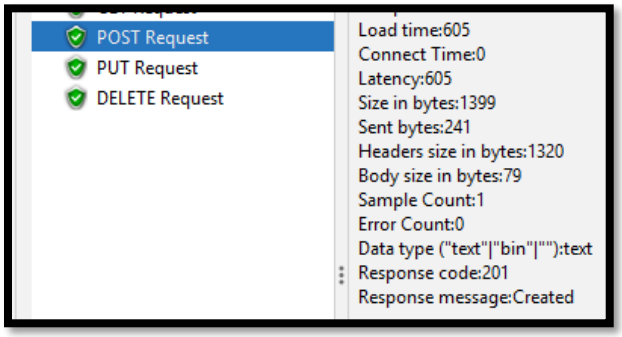
Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/...	Sent KB/sec	Avg. Bytes
PUT Request	1	433	433	433	0.00	0.00%	2.3/sec	2.87	0.57	1274.0
POST Request	1	468	468	468	0.00	0.00%	2.1/sec	2.92	0.50	1399.0
GET Request	2	582	448	717	134.50	0.00%	24.9/hour	0.01	0.00	1515.0
DELETE Requ...	1	241	241	241	0.00	0.00%	4.1/sec	4.81	0.90	1187.0
TOTAL	5	461	241	717	151.62	0.00%	1.0/min	0.02	0.00	1378.0



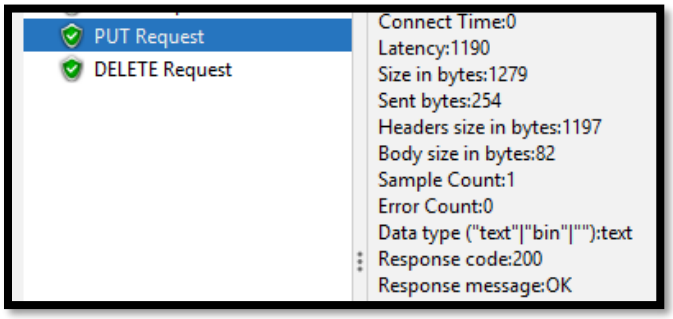

```
GET Request Code:        200
PUT Request Code:        200
POST Request Code:       201
DELETE Request Code:     200
```



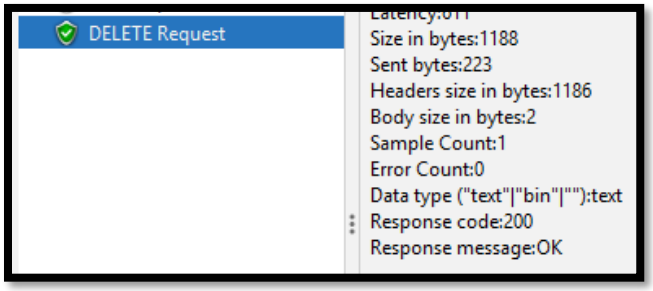
GET



POST



PUT



DELETE

Part C — Assertions

Add three types of assertions to the GET /posts/1 request to automatically validate the server response.

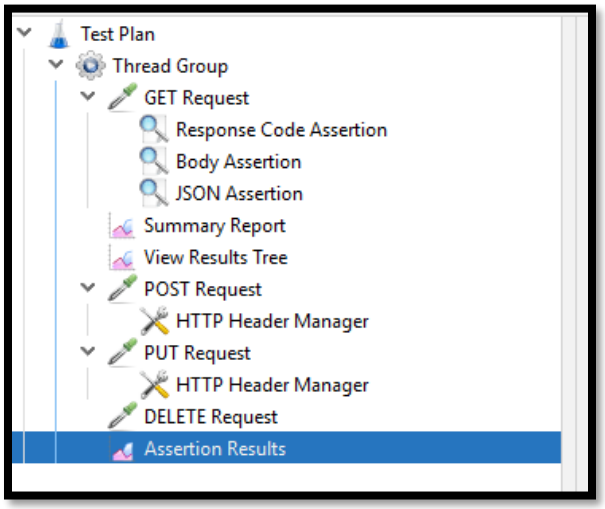
1. Response Code Assertion — verifies the HTTP status code is 200
2. Response Body Assertion — verifies the response body contains the word 'userId'
3. JSON Assertion — verifies the JSON field $.userId equals the value 1

```
Assertion 1 — Response Code:
  Right-click GET sampler → Add → Assertions → Response Assertion
  Field to Test: Response Code   Pattern: Equals   Value: 200

Assertion 2 — Response Body Contains Text:
  Add another Response Assertion
  Field to Test: Response Body   Pattern: Contains   Value: userId

Assertion 3 — JSON Path Assertion:
  Right-click GET sampler → Add → Assertions → JSON Assertion
  JSON Path: $.userId    Expected Value: 1    Check 'Additionally assert value'
```

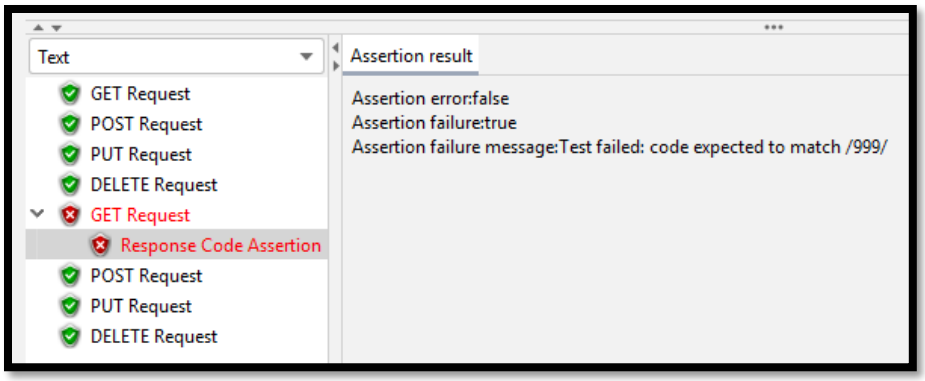
4. Intentional failure test: Change the Response Code assertion value to 999, run, and observe the RED failure. Then restore it back to 200 and confirm GREEN again

Task 1 — Deliverables:

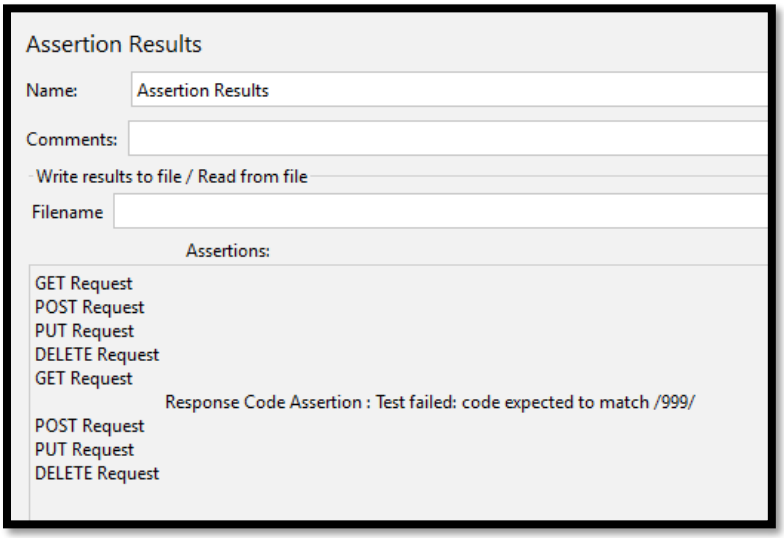
[ ]	Labelled screenshot of JMeter GUI identifying all 5 components
[ ]	Screenshot of View Results Tree showing the GET request passing (GREEN)
[ ]	Screenshot of all 4 HTTP methods in View Results Tree — all GREEN
[ ]	Screenshot of all 3 assertions passing on the GET request
[ ]	Screenshot of the intentional assertion failure showing RED result with error message



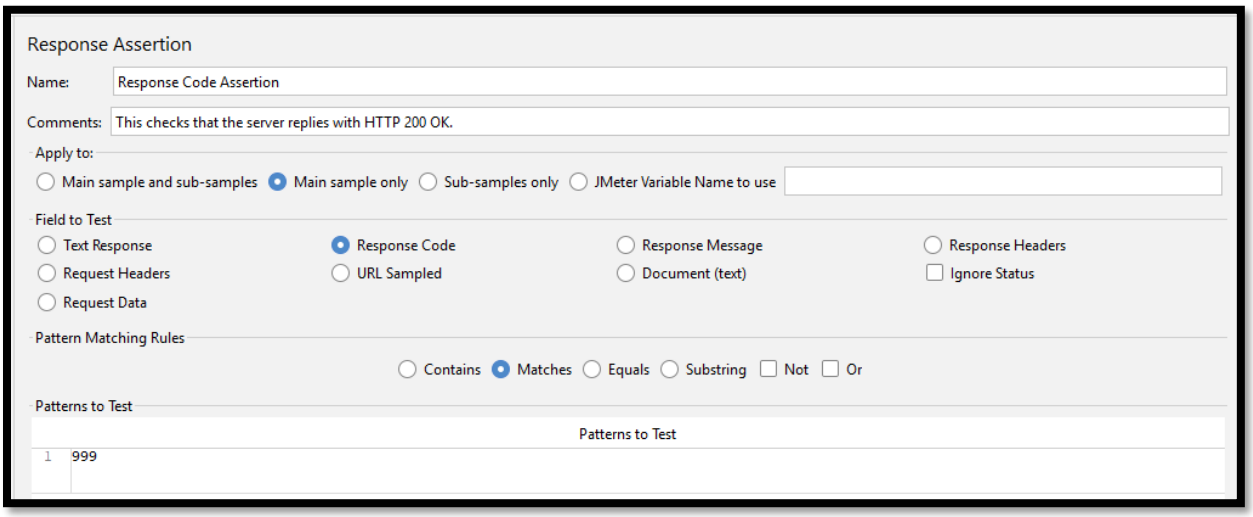
TREE



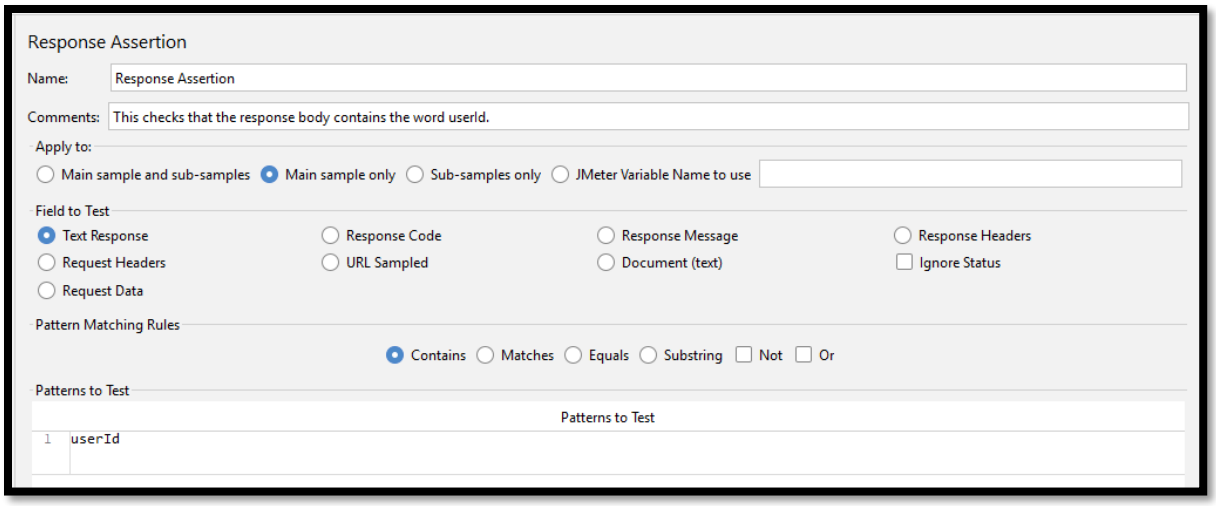
Success and failure test by changing response code to 999.



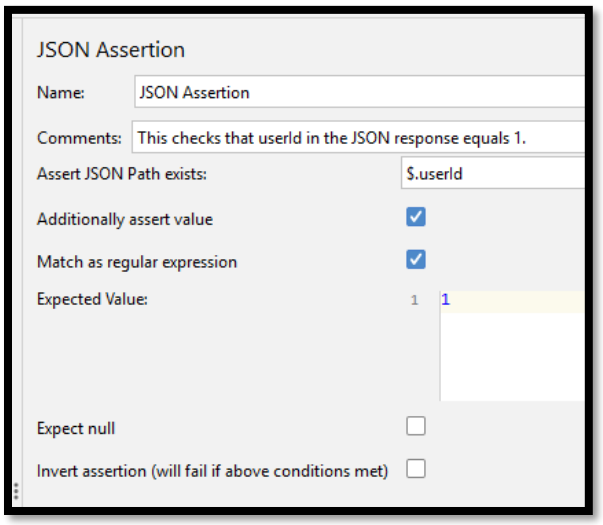
failure test in assertion results matching code 999



Response Code Assertion



Body Assertion



JSON Assertion

TASK 2 Performance Testing — Load, Stress & Data-Driven

Covers: Load Test · Service Level Agreement Evaluation · Stress Test · CSV Parameterization · Correlation

This is the core of JMeter — simulating real users under load. You will run a formal load test against defined performance targets, find the system's breaking point with a stress test, and use CSV files to send different data for each virtual user.

Part A — Load Test with Service Level Agreement Evaluation

A Service Level Agreement (Service Level Agreement) defines the minimum performance standards a system must meet. You will run a load test and check whether the application meets all four targets below.

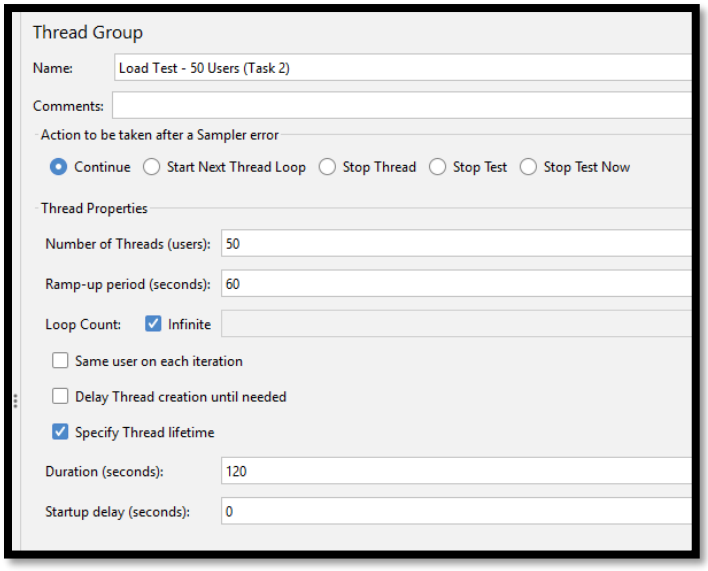
PERFORMANCE TARGETS — Service Level Agreement

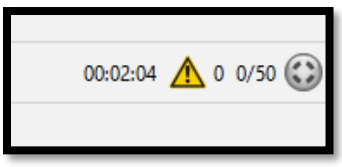
Average Response Time → must be UNDER 2000 milliseconds
90th Percentile → 90% of all requests must complete in UNDER 3000 milliseconds
Error Rate → must be UNDER 1% of total requests
Throughput → server must handle at LEAST 5 requests per second

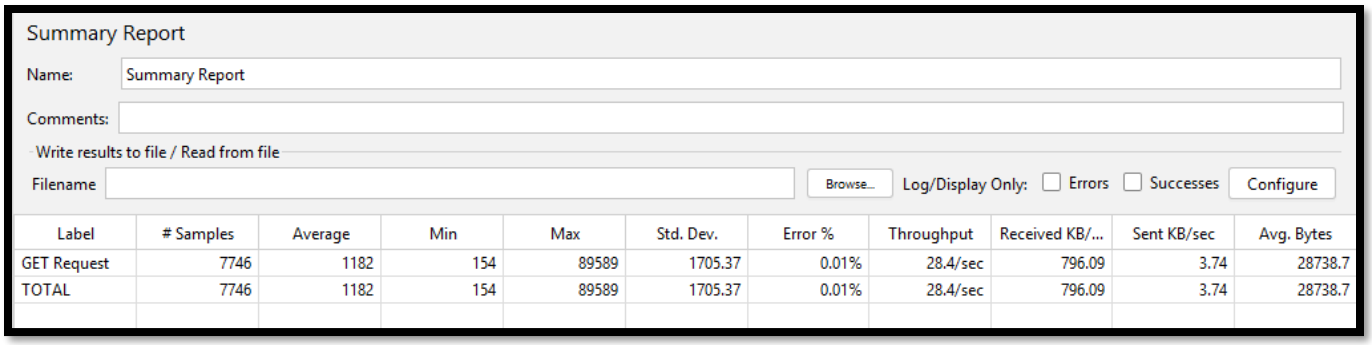
1. Target endpoint: GET https://jsonplaceholder.typicode.com/posts
2. Configure Thread Group:

```
Number of Threads (users):  50
Ramp-Up Period:             30 seconds
Loop Count:                 10
Scheduler: checked          Duration: 120 seconds  (run for 2 minutes)
```

3. Add these listeners: Aggregate Report, Response Time Graph, Summary Report
4. Run the full 2-minute test. Record all 4 metrics from the Aggregate Report
5. Write a PASS or FAIL verdict for each metric against the Service Level Agreement targets above







Summary Report

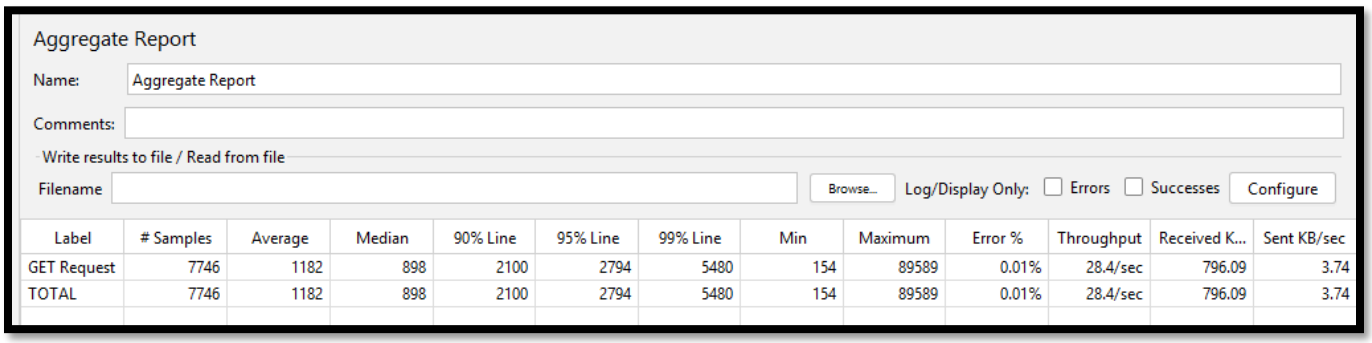
Name: Summary Report

Comments:

Write results to file / Read from file

Filename Browse... Log/Display Only: Errors Successes Configure

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/...	Sent KB/sec	Avg. Bytes
GET Request	7746	1182	154	89589	1705.37	0.01%	28.4/sec	796.09	3.74	28738.7
TOTAL	7746	1182	154	89589	1705.37	0.01%	28.4/sec	796.09	3.74	28738.7



Aggregate Report

Name: Aggregate Report

Comments:

Write results to file / Read from file

Filename Browse... Log/Display Only: Errors Successes Configure

Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received K...	Sent KB/sec
GET Request	7746	1182	898	2100	2794	5480	154	89589	0.01%	28.4/sec	796.09	3.74
TOTAL	7746	1182	898	2100	2794	5480	154	89589	0.01%	28.4/sec	796.09	3.74

Metric	SLA Target	Pass/Fail?
Average Response Time	< 2000ms	☑
Error Rate	< 1%	☑
Throughput	As high as possible	28.4/sec
90th Percentile	< 3000ms	☑

Part B — Stress Test: Find the Breaking Point

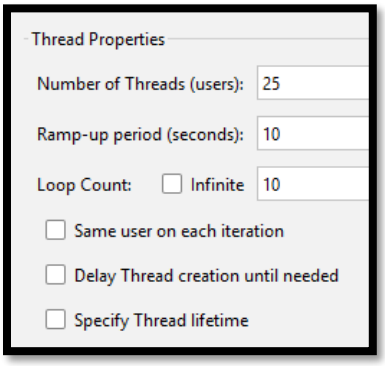
SERVICE LEVEL AGREEMENT vs STRESS TEST
Load Test: Verify system handles EXPECTED traffic within the Service Level Agreement targets
Stress Test: Push BEYOND normal limits to find WHEN the system starts failing
The user count where error rate first exceeds 5% is called the breaking point

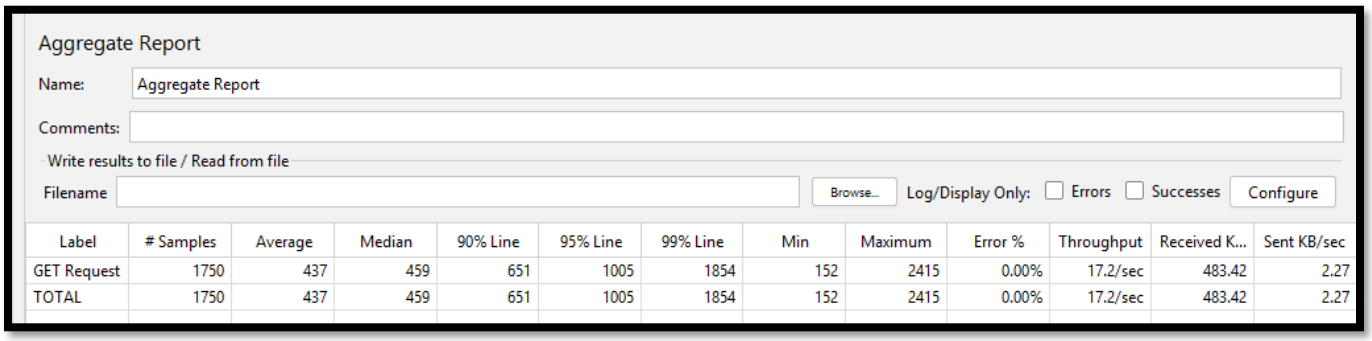
6. Run the same endpoint 4 times with increasing users. Stop when error rate exceeds 5%:

```
Run 1:   25 users,  ramp-up 10s,  loop 10  → record error %
Run 2:   75 users,  ramp-up 15s,  loop 10  → record error %
Run 3:  150 users,  ramp-up 25s,  loop 10  → record error %
Run 4:  300 users,  ramp-up 40s,  loop 10  → record error %
```

7. Identify the breaking point — at how many users did error rate first exceed 5%?

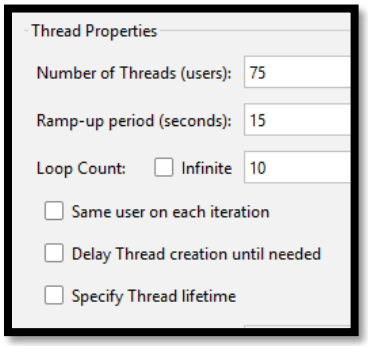
RUN 1

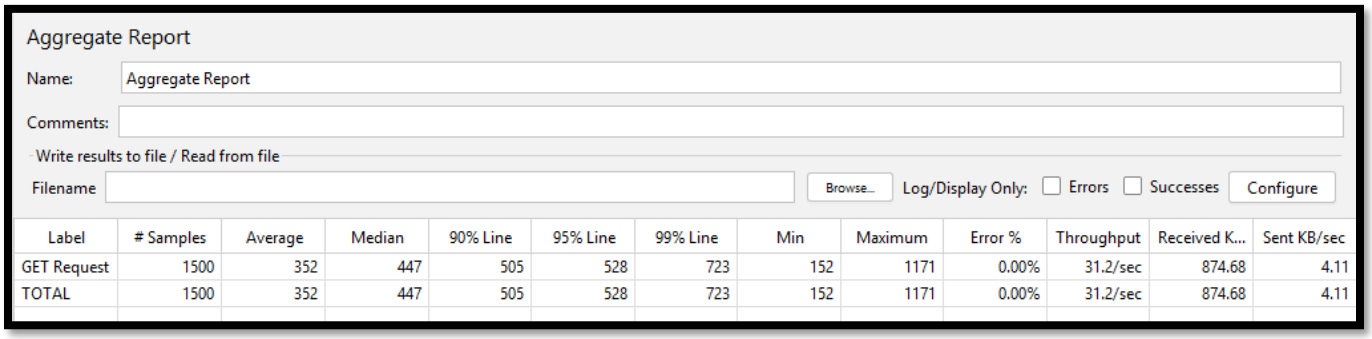




Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received K...	Sent KB/sec
GET Request	1750	437	459	651	1005	1854	152	2415	0.00%	17.2/sec	483.42	2.27
TOTAL	1750	437	459	651	1005	1854	152	2415	0.00%	17.2/sec	483.42	2.27

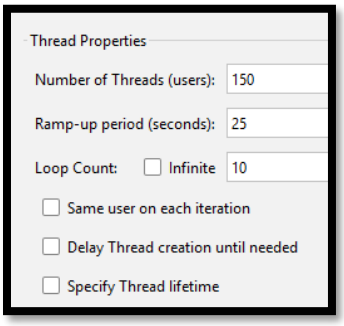
RUN 2

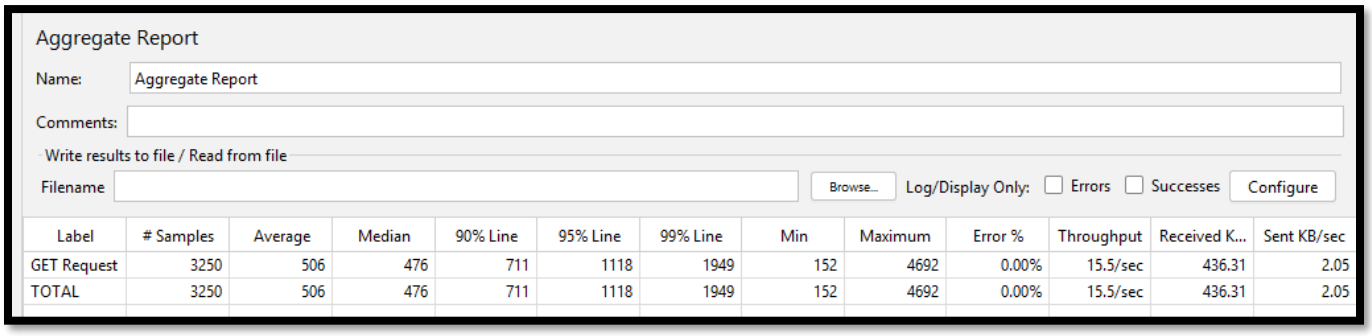




Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received K...	Sent KB/sec
GET Request	1500	352	447	505	528	723	152	1171	0.00%	31.2/sec	874.68	4.11
TOTAL	1500	352	447	505	528	723	152	1171	0.00%	31.2/sec	874.68	4.11

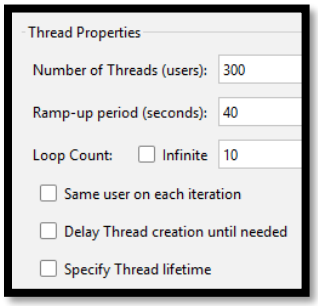
RUN 3

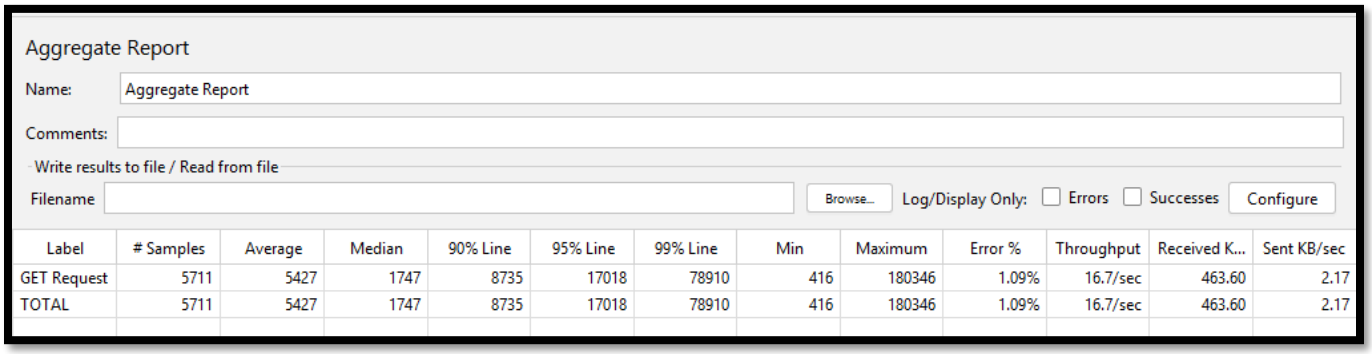




Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received K...	Sent KB/sec
GET Request	3250	506	476	711	1118	1949	152	4692	0.00%	15.5/sec	436.31	2.05
TOTAL	3250	506	476	711	1118	1949	152	4692	0.00%	15.5/sec	436.31	2.05

RUN 4





Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received K...	Sent KB/sec
GET Request	5711	5427	1747	8735	17018	78910	416	180346	1.09%	16.7/sec	463.60	2.17
TOTAL	5711	5427	1747	8735	17018	78910	416	180346	1.09%	16.7/sec	463.60	2.17

Run	Users	Avg Response (ms)	90% Line (ms)	Error %	Throughput
1	25	437	651	0.00	17.2/sec
2	75	352	505	0.00	31.2/sec
3	150	506	711	0.00	15.5/sec
4	300	5427	8735	1.09	16.7/sec

Part C — CSV Data-Driven Testing & Correlation

Real tests send different data for each virtual user instead of the same hardcoded value. This part covers CSV parameterization and passing data between two chained requests (called correlation).

8. Create a file named test_data.csv with this content:

```
post_id,expected_code
1,200
10,200
50,200
100,200
999,404
```

9. Add CSV Data Set Config to Thread Group (5 users, loop 1):

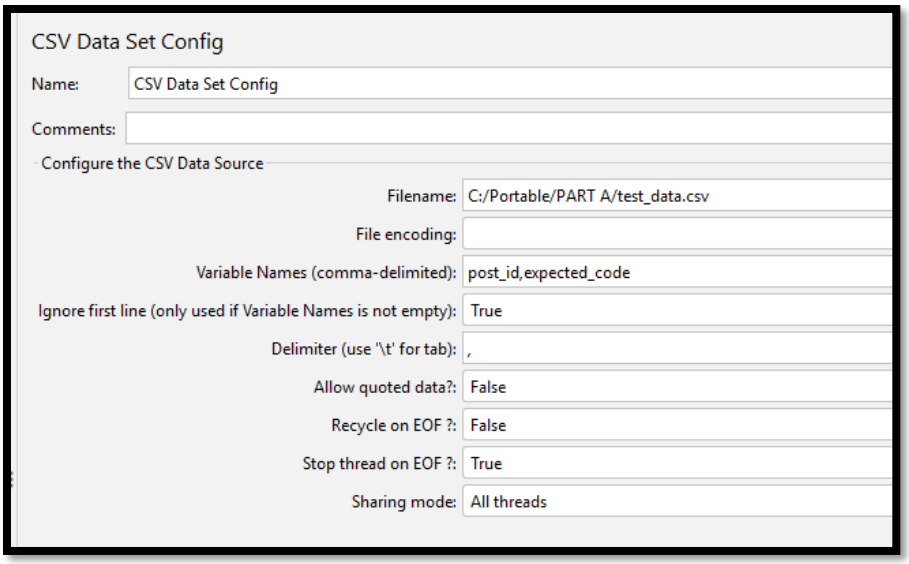
```
Right-click Thread Group → Add → Config Element → CSV Data Set Config
Filename:            full path to your test_data.csv
Variable Names:      post_id,expected_code
Delimiter:           ,
Stop thread on EOF:  True
```

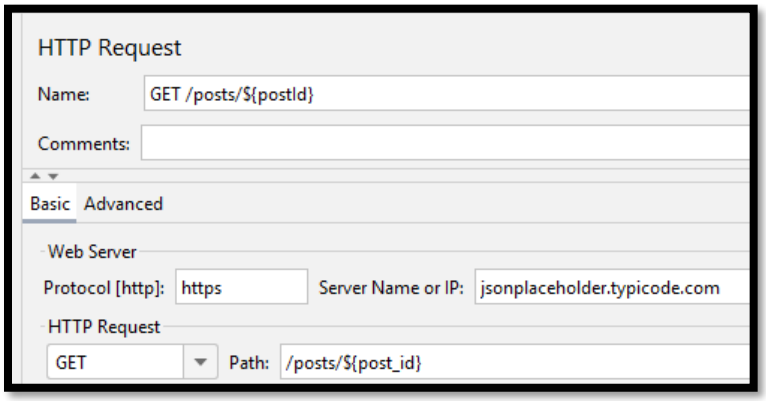
10. Set HTTP Request path to /posts/${post_id} and add Response Assertion using ${expected_code}
11. Add a JSON Extractor to the HTTP Request to extract the returned post ID:

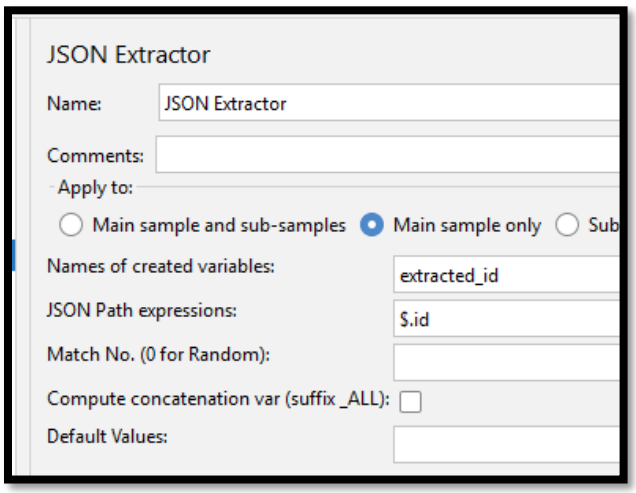
```
Right-click HTTP Request → Add → Post Processors → JSON Extractor
Variable name:   extracted_id
JSON Path:       $.id

Add a second HTTP Request using the extracted value:
Path: /posts/${extracted_id}/comments

Verify in View Results Tree that Request 2 URL contains the correct extracted ID
```

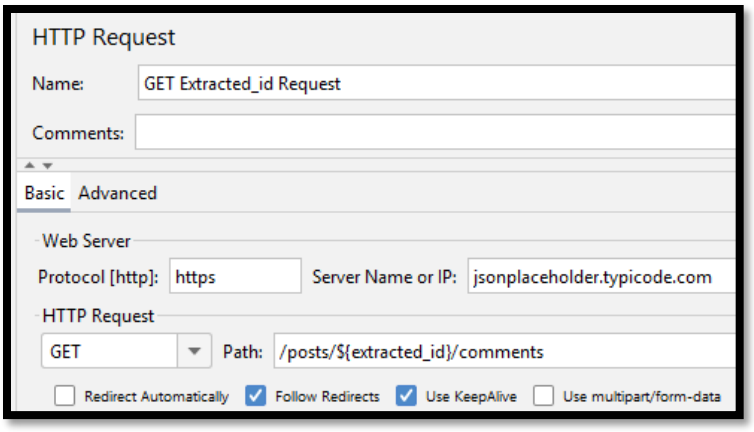
Task 2 — Deliverables:

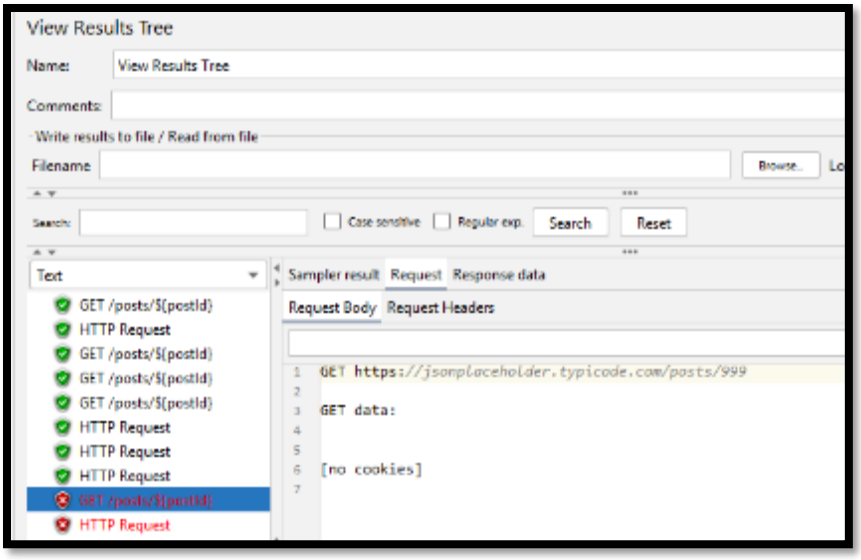
[ ]	Screenshot of Aggregate Report from the 50-user load test
[ ]	Service Level Agreement table in your report: metric | measured value | target | PASS or FAIL
[ ]	Screenshot of Response Time Graph showing the full 2-minute run
[ ]	Stress test table: all 4 runs with user count and error percentage
[ ]	Written answer: What was the breaking point and at how many users did it occur?
[ ]	Screenshot of CSV Data Set Config settings
[ ]	Screenshot of View Results Tree showing 5 users hitting different endpoints
[ ]	Screenshot of View Results Tree showing Request 2 URL using the extracted ID
[ ]	Written answer: What is correlation and when is it needed?


CSV Data Set Config
Name: CSV Data Set Config
Comments:
Configure the CSV Data Source
Filename: C:/Portable/PART A/test_data.csv
File encoding:
Variable Names (comma-delimited): post_id,expected_code
Ignore first line (only used if Variable Names is not empty): True
Delimiter (use '\t' for tab): ,
Allow quoted data?: False
Recycle on EOF ?: False
Stop thread on EOF ?: True
Sharing mode: All threads

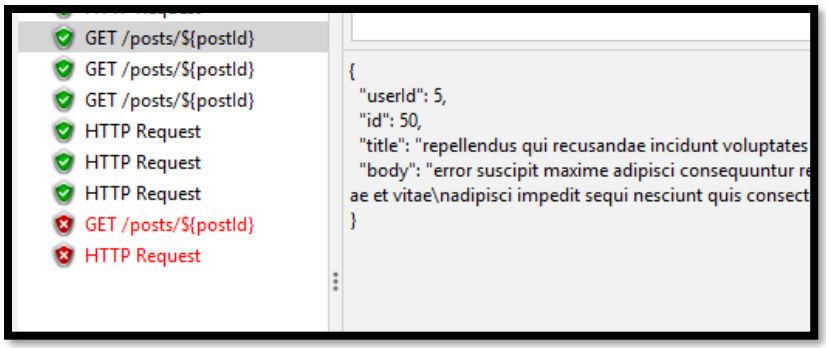

HTTP Request
Name: GET /posts/${postId}
Comments:
Basic Advanced
Web Server
Protocol [http]: https
Server Name or IP: jsonplaceholder.typicode.com
HTTP Request
GET
Path: /posts/${post_id}

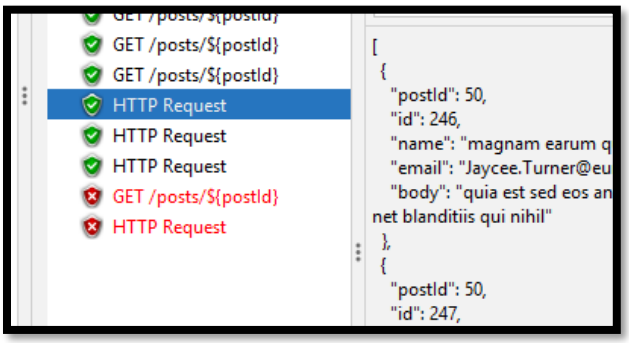

JSON Extractor
Name: JSON Extractor
Comments:
Apply to:
Main sample and sub-samples
Main sample only
Sub
Names of created variables: extracted_id
JSON Path expressions: $.id
Match No. (0 for Random):
Compute concatenation var (suffix _ALL):
Default Values:



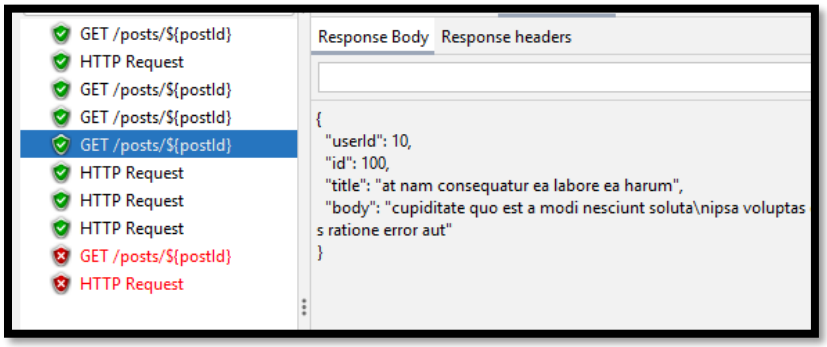


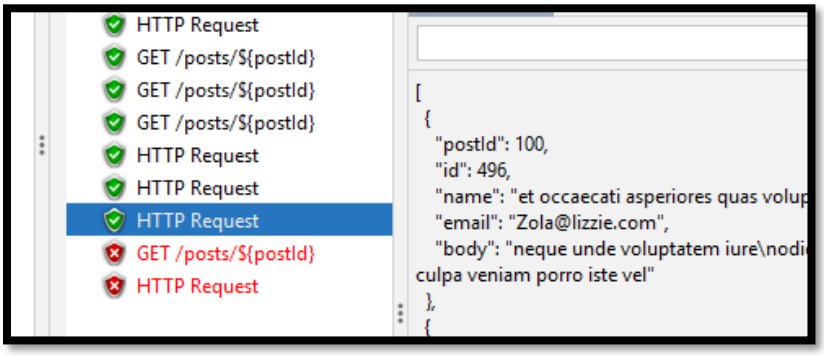
Followings 2 are failed due to there is no id ***999*** exist on server side





HIT ID 50





HIT ID 100

TASK 3 Professional JMeter — CI/CD Pipeline Integration

· GitHub Actions Automation

This task covers professional-grade JMeter usage as practiced in real software teams. You will run your test from the command line, generate a full HTML performance dashboard, and set up an automated pipeline on GitHub so your test runs every time code is pushed.

Non-GUI Reports & GitHub Actions CI/CD

IMPORTANT RULE FOR REAL PROJECTS

Never run actual load tests using the JMeter GUI — it uses extra CPU and memory which distorts results.
Always run performance tests from the command line (called non-GUI or headless mode).
Rule to remember: Use the GUI only to BUILD the test plan. Always RUN from the command line.

Step 1 — Run JMeter from the Command Line

12. Save your Task 2 load test as load_test.jmx
13. Open a terminal or command prompt and run the following command:

```
# Windows:
cd C:\apache-jmeter-5.6\bin
jmeter -n -t C:\path\load_test.jmx -l C:\results\results.jtl -e -o C:\reports\

# Linux / Mac:
cd /opt/jmeter/bin
./jmeter.sh -n -t ~/load_test.jmx -l ~/results/results.jtl -e -o ~/reports/

What each flag means:
  -n   =  Non-GUI (headless) mode
  -t   =  Path to your .jmx test plan file
  -l   =  Path to save the raw results (.jtl file)
  -e   =  Generate HTML report after test finishes
  -o   =  Output folder for HTML report (must be empty before running)
```

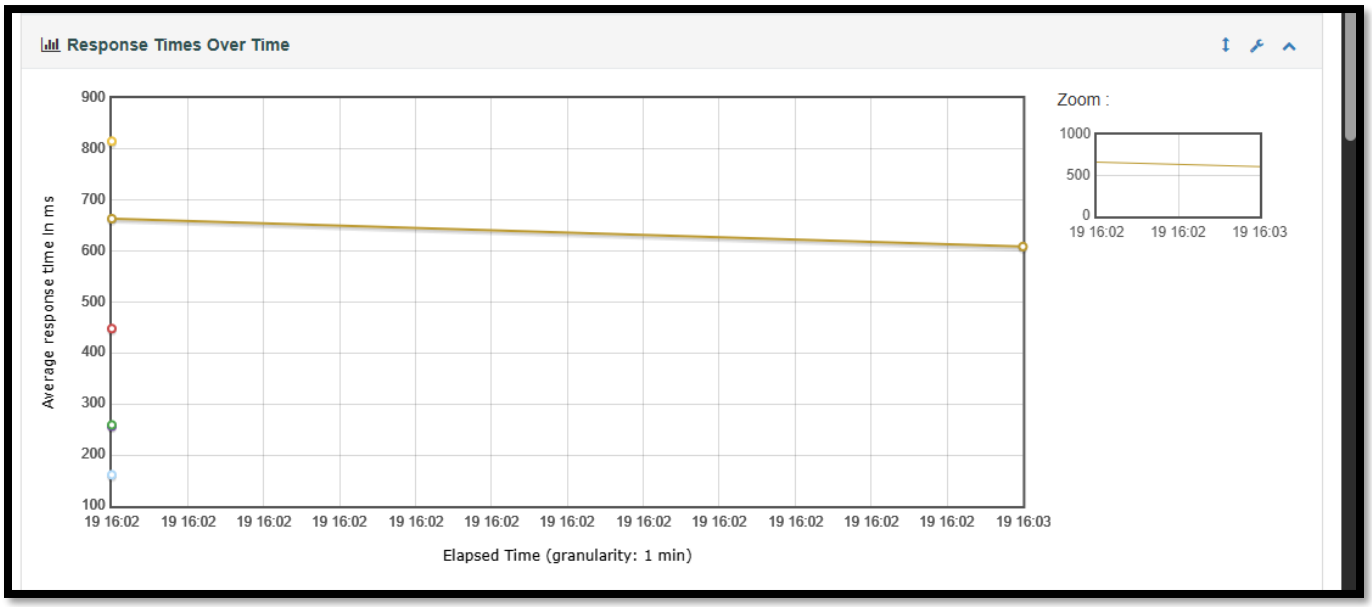
14. Open the generated report: browse to your reports folder and open index.html in a browser
15. Explore these 3 sections of the HTML dashboard and describe each one in your report:
    - Response Time Over Time chart — how did response time change during the test?
    - Throughput Over Time chart — how many requests per second at each moment?
    - Response Time Percentiles — what do the P90 and P99 values mean?

```
┌─[ SWEET ]─[ TUESDAY 19/05/2026 - 3:54:14 PM ]─[ C:\Portable\PART A ]
└─[ UMAIR 🤚 PowerShell ] :]  jmeter -n -t 'test_plan.jmx' -l results.jtl -e -o reports
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARN StatusConsoleListener The use of package scanning to locate plugins is deprecated and will be removed in a future release
WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe::objectFieldOffset has been called by com.thoughtworks.xstream.converters.reflection.SunUnsafeReflectionProvider (file:/C:/
Portable/apache-jmeter/lib/xstream-1.4.20.jar)
WARNING: Please consider reporting this to the maintainers of class com.thoughtworks.xstream.converters.reflection.SunUnsafeReflectionProvider
WARNING: sun.misc.Unsafe::objectFieldOffset will be removed in a future release
Creating summariser <summary>
Created the tree successfully using test_plan.jmx
Starting standalone test @ 2026 May 19 15:54:26 GMT+05:00 (1779188066027)
Waiting for possible Shutdown/StopTestNow/HeapDump/ThreadDump message on port 4445
summary +     85 in 00:00:04 =   22.0/s Avg:   525 Min:   143 Max:  1111 Err:     2 (2.35%) Active: 23 Started: 29 Finished: 6
summary +   1429 in 00:00:28 =   50.5/s Avg:   576 Min:   426 Max:  3919 Err:     0 (0.00%) Active: 0 Started: 156 Finished: 156
summary =   1514 in 00:00:32 =   47.0/s Avg:   573 Min:   143 Max:  3919 Err:     2 (0.13%)
Tidying up ...    @ 2026 May 19 15:54:58 GMT+05:00 (1779188098357)
... end of run
```

Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent
Total	**1514**	**2**	**0.13%**	**573.88**	**143**	**3919**	**499.00**	**658.00**	**1064.75**	**1967.65**	**47.56**	**1323.29**	**6.28**
DELETE Request	1	0	0.00%	285.00	285	285	285.00	285.00	285.00	285.00	3.51	4.07	0.76
GET /posts/${postId}	5	1	20.00%	776.20	492	1111	748.00	1111.00	1111.00	1111.00	4.16	5.85	0.56
GET Extracted_id Request	5	1	20.00%	148.00	143	153	148.00	153.00	153.00	153.00	24.39	58.38	3.49
GET Request	1501	0	0.00%	575.11	426	3919	499.00	653.80	1059.20	1968.82	47.15	1322.57	6.22
POST Request	1	0	0.00%	269.00	269	269	269.00	269.00	269.00	269.00	3.72	5.08	0.87
PUT Request	1	0	0.00%	439.00	439	439	439.00	439.00	439.00	439.00	2.28	2.83	0.57

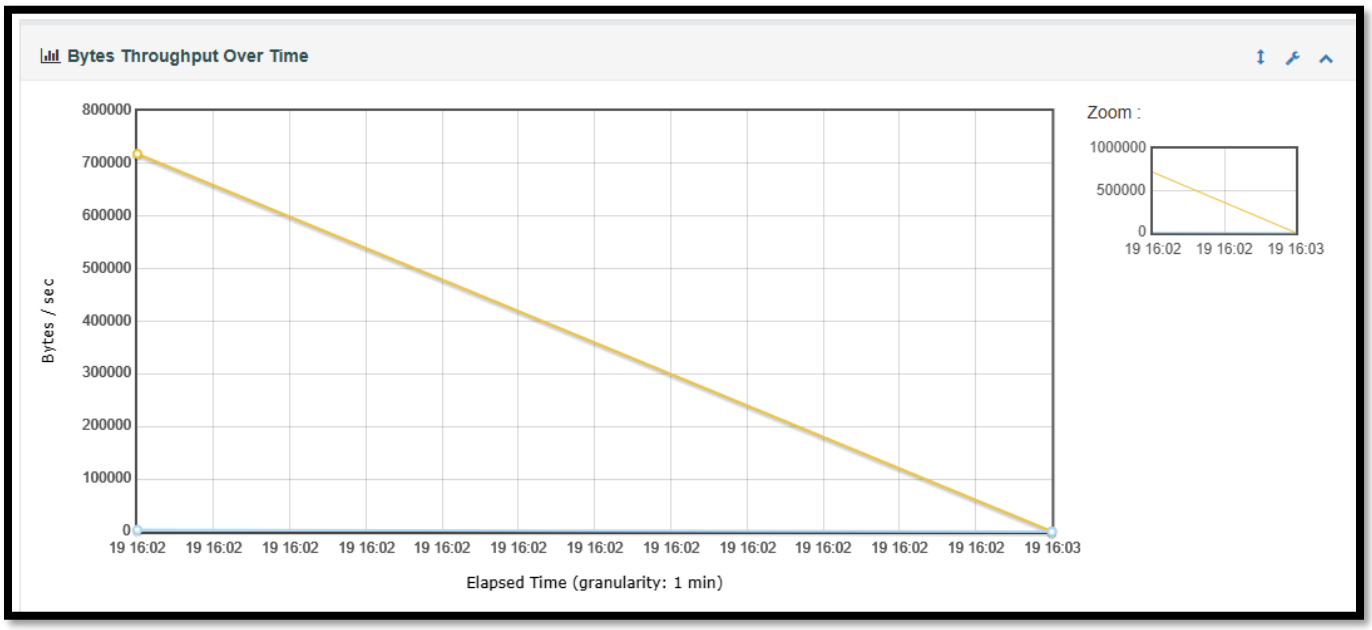
Response Time Over Time

This chart shows how long requests took (in ms) as the test ran. A flat line means the server was stable. A rising line means it slowed down under load. A sudden spike means the server hit its breaking point.



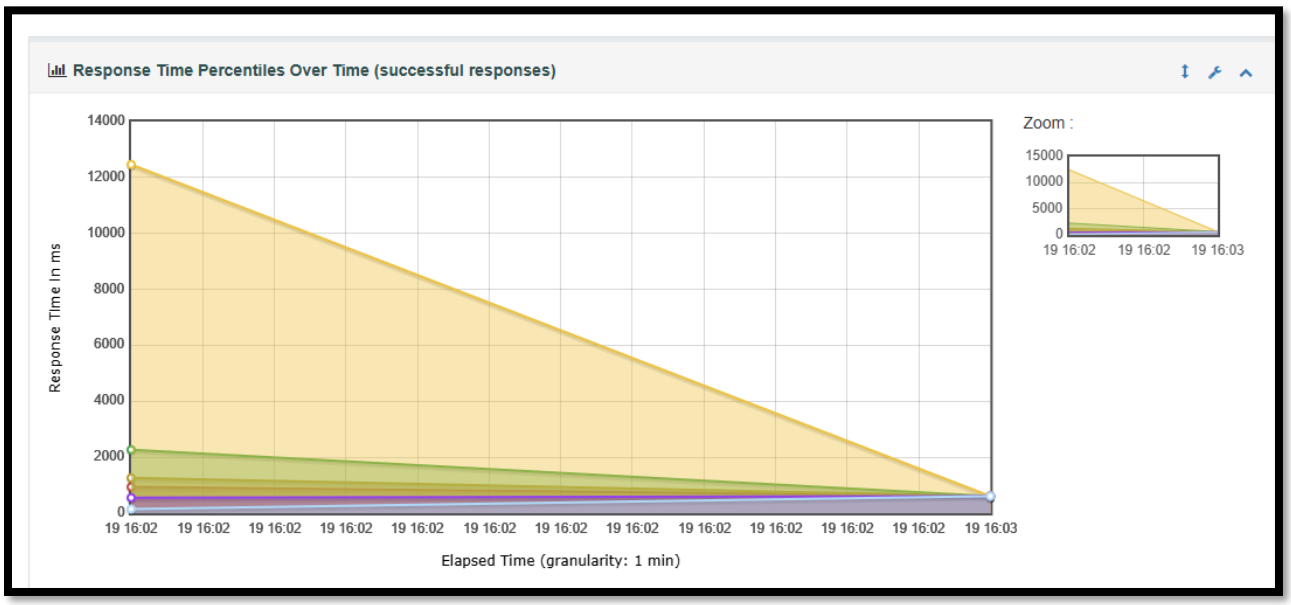
Throughput Over Time

This chart shows how many requests per second the server handled at each moment. It usually rises during ramp-up then flattens out. If throughput drops while users increase, the server is overwhelmed.



P90 & P99 Percentiles

P90 means 90% of users got a response within that time. P99 means 99% of users got a response within that time. The remaining 1% are the slowest outliers. Lower P90/P99 values mean a better, more consistent experience for almost all users.



Step 2 — CI/CD Pipeline with GitHub Actions

Automate your JMeter test so it runs automatically every time code is pushed to GitHub. This is exactly how real Quality Assurance teams catch performance problems before they reach users.

16. Create a free GitHub account at github.com if you do not already have one
17. Create a new public repository named jmeter-semester-project
18. Upload your load_test.jmx file into a folder called tests/ inside the repository
19. Create a new file at this path in the repository: .github/workflows/jmeter.yml
20. Add the following content to the workflow file:

```
name: JMeter Performance Tests

on:
  push:
    branches: [ main ]

jobs:
  performance-test:
    runs-on: ubuntu-latest
    steps:

      - name: Checkout repository
        uses: actions/checkout@v3

      - name: Set up Java 11
        uses: actions/setup-java@v3
        with:
          java-version: '11'
          distribution: 'temurin'

      - name: Download Apache JMeter
        run: |
          wget -q https://downloads.apache.org/jmeter/binaries/apache-jmeter-
5.6.3.tgz
          tar -xzf apache-jmeter-5.6.3.tgz && mv apache-jmeter-5.6.3 jmeter

      - name: Run Performance Tests
        run: |
          mkdir -p results reports
          jmeter/bin/jmeter -n \
            -t tests/load_test.jmx \
            -l results/results.jtl \
            -e -o reports/

      - name: Upload HTML Report as Artifact
        uses: actions/upload-artifact@v3
        if: always()
        with:
          name: jmeter-report
          path: reports/
```

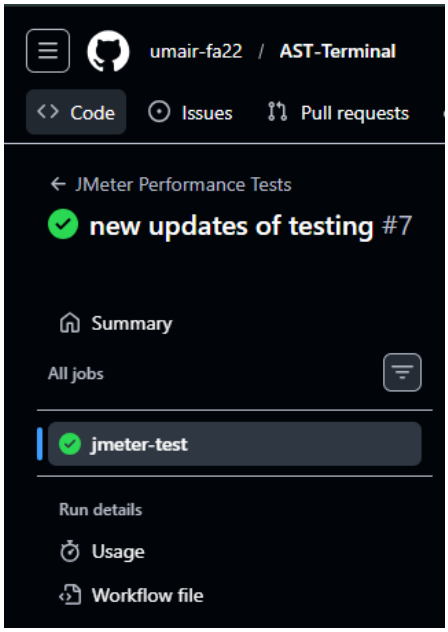
21. Commit and push all files to GitHub
22. Go to your repository on GitHub → click the Actions tab → watch the pipeline run automatically
23. After it completes → click the finished run → download the jmeter-report artifacts

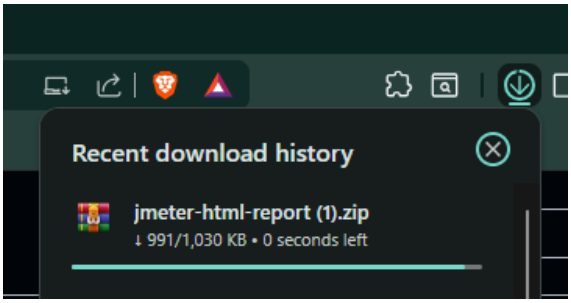
Task 3 — Deliverables:

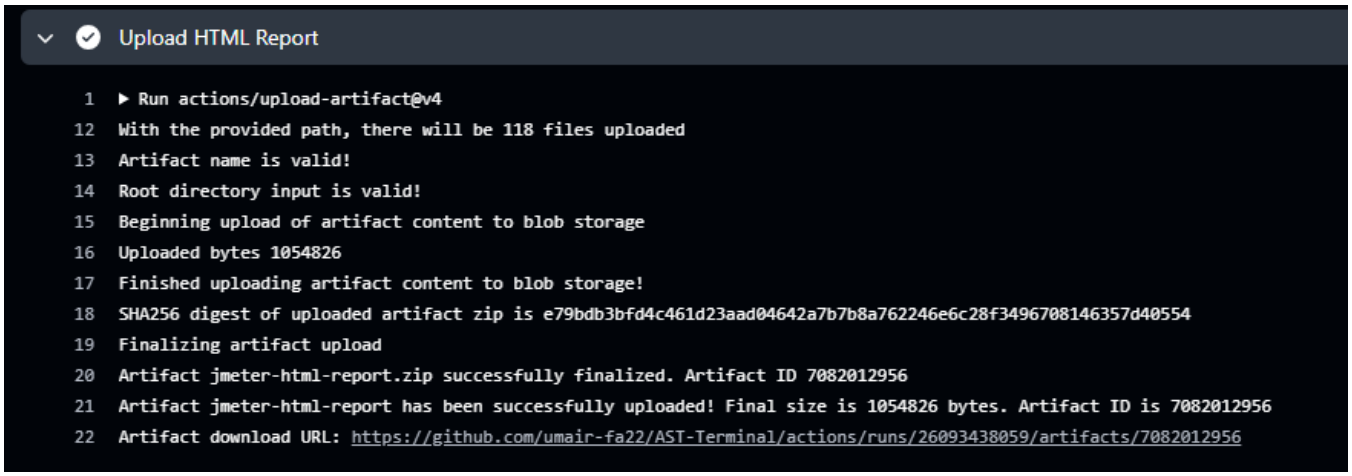
[ ]	Screenshot of the terminal showing JMeter running in non-GUI mode
[ ]	Screenshot of the HTML Dashboard index.html open in your browser
[ ]	Screenshot of the Statistics Table from the HTML dashboard
[ ]	Screenshot of the Response Time Over Time chart from the dashboard
[ ]	Written explanation (2–3 sentences): What does the P90 value in your report mean?
[ ]	GitHub repository link — share this with your instructor
[ ]	Screenshot of GitHub Actions pipeline completing successfully (green checkmark)
[ ]	Screenshot of the downloaded jmeter-report artifact from GitHub Actions

No local server needed — all tasks use free public APIs. Task 3 requires a free GitHub account. Each student's .jtl result file has unique timing values for plagiarism checking.

GitHub repository link: https://github.com/umair-fa22/AST-Terminal.git







Submission Requirements box:
Submit one PDF report with screenshots and written answers, plus your .jmx test plan file. Include your Name, Student ID, and Date on the cover page.

Sample tables for Students [Will assist in VIVA]

Metric	Measured Value	Required Target	Pass / Fail	Notes
Avg Response Time	_____	_____	_____	_____
90th Percentile	_____	_____	_____	_____
Error Rate	_____	_____	_____	_____
Throughput	_____	_____	_____	_____

Results Table to Complete:

Test Run	Avg Response (ms)	Throughput (req/s)	Error Rate %	Max Response (ms)
1 User	_____	_____	_____	_____
10 Users	_____	_____	_____	_____
50 Users	_____	_____	_____	_____